

Trilogue 現状報告書（教授向け） — Phase 2-B: マルチエージェント対話（サーバー側オーケストレーション）

1. 報告概要

先日のミーティングで合意した「マルチエージェント対話システムの構築」について、その中核となるサーバー側での会話制御（オーケストレーション）を実装し、LabVIEW実機での動作確認まで完了した。

あわせて、ご指摘いただいていた「そのサーバーが別のPCで動くかどうかの検証」についても、Docker化により Node.js・Gitを一切インストールしない環境で起動できることを実証した。

本報告の要点は次の3点である。

- LabVIEWから1回投げるだけで、複数のAIが順番に発言し、会話全体がまとめて返る構成を実装した
- 各AIは直前までの発言を読んだうえで発言するため、並列回答ではなく会話が成立している
- 発言順を決める処理を独立させ、次段階のファシリテーターAI導入に接続できる形にした

2. 段階的な実装計画における位置づけ

マルチエージェント化は3段階で進める計画としており、今回は段階Bが完了した。

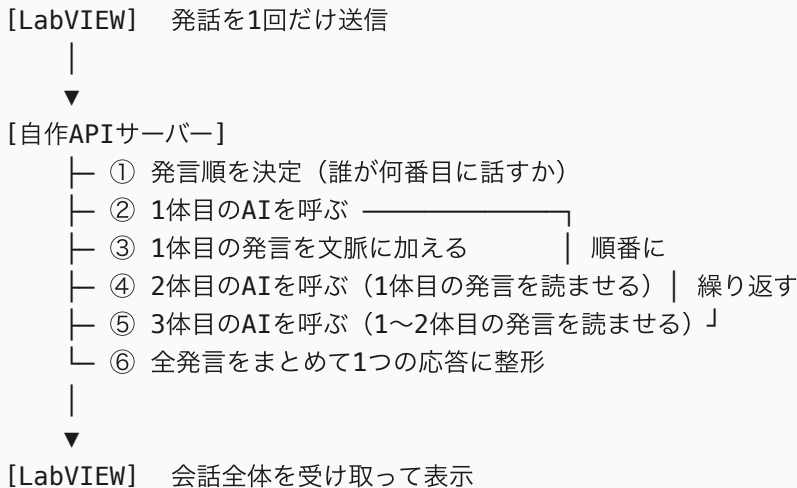
段階	内容	状態
A	LabVIEW主導で、AIを1体ずつ順番に呼ぶ（最小の「AI同士の会話」）	既存基盤で実現可能
B	サーバー側オーケストレーション：1リクエストで複数AIの応答をまとめて返す	今回完了
C	ファシリテーター（司会）AI+発言順のポリシー化（偏り制御・質問ガイド）	次の実装対象

段階Aは「LabVIEW側が何度も呼ぶ」方式で、会話が増えるほどLabVIEW側の作りが複雑になる。そこで段階Bでは、制御の責任をサーバー側に移した。これによりLabVIEW側は往復1回のままで、実装量を増やさずに複数AIの会話を扱える。

3. 今回実装した内容

3.1 複数AIの応答をまとめて返す仕組み

LabVIEWからは、患者の発話を1回送るだけでよい。サーバーは以下を内部で行い、結果を1つにまとめて返す。



3.2 「並列回答」ではなく「会話」にした点

各AIには**直前までの発言が文脈として渡る**設計とした。会話ログ上では発言者を区別できる形で記録し、各AIには「これは複数のAIが順番に発言する場である」「既出の意見をそのまま繰り返さない」ことを役割定義とあわせて伝えている。

その結果、後続のAIが前の発言を受けて別の視点を述べる挙動が実際に確認できた（第5章の実測参照）。

3.3 LabVIEW側の実装を増やさないための工夫

複数AIの発言は、**配列と1本に連結した文字列**の両方の形で返す。

- LabVIEW側は **成否フラグと連結文字列の2つだけ**を見れば会話を表示できる
- 配列は無視しても動作するため、既存VIとほぼ同じ作りで対応できる
- AIごとに個別表示したい場合のみ、配列を解析すればよい

実際、既存の単体AI用VIを複製し、**変更箇所は4点のみ**で複数AI版が完成した。

3.4 堅牢性への配慮

- **部分成功の許容**：1体のAIが失敗しても、残りのAIの発言は続行し、取れた分は返す。失敗した発言は個別に理由を記録するため、どこで何が起きたか追跡できる。
- **発言数の上限**：1発言ごとにLLMを1回呼ぶため、応答時間と利用料が膨らまないようサーバー側で総発言数に上限を設けた。

- ・ **自動テスト**：発言順・文脈の受け渡し・部分成功・上限超過の各挙動を、外部通信を行わない自動テストで検証している。

4. 段階C（ファシリテーターAI）への接続

今回の実装では、「発言順を決める処理」と「実際にAIを呼ぶ処理」を意図的に分離した。

現在の発言順は単純な順番制御だが、この決定部分だけをAIの判断に差し替えれば、そのままファシリテーターAIになる構造としてある。

要素	現在（段階B）	段階Cでの拡張
発言順の決定	指定された順に巡回	司会AIが文脈から次の発話者を指名
役割（ペルソナ）	定義を差し替え可能な形で保持	場面に応じて動的に切り替え
会話フロー	1巡ごとに完結	質問の投げ方・打ち切り条件をポリシー化
偏りの制御	全員が均等に発言	発言量・視点の偏りを司会AIが調整

拡張時に作り直しが発生しないよう、あらかじめ境界を設けた形である。

5. 動作確認結果

5.1 LabVIEW実機での往復（研究室Windows機 ↔ サーバー）

- ・ 研究室のWindows機で動作するLabVIEWから、同一LAN経由でサーバーへ送信
- ・ 1回のリクエストで3体のAIが順番に発言し、会話全体がLabVIEW画面に表示された
- ・ サーバログ上も正常応答（HTTP 200）を確認、所要 約4.7秒

5.2 3体のAIによる応答内容（実測）

入力（患者の感想を想定）：

リハビリのトレーニングを終えました。指が少し動きやすくなった気がして、続けられそうです。

出力（3体が順番に発言。所要 6.3秒）：

順	AI	発言の要旨
1	Alpha	「素晴らしいですね。少しずつでも改善を実感できることは励みになります。継続が大切です」
2	Beta	「良い兆候です。ただし無理をしないよう、専門家の指導を仰ぎ続けることも大切です」

順	AI	発言の要旨
3	Gamma	「その体験を他の挑戦にも生かすことで、日常生活にも新たな可能性が広がります」

注目点：2体目が「ただし」と受けて慎重な補足に回り、3体目はさらに別の角度から広げている。3体と同じ内容を繰り返しておらず、**互いの発言を踏まえた会話**になっていることが確認できる。

将来的に、ここへ専門領域ごとの役割を割り当てることで、「複数の視点から患者の状態を多角的に評価する」構成へ発展させられる。

5.3 応答時間の実測

条件	所要時間
AI 2体	約1.5～4.7秒
AI 3体	約6.3秒

- 所要時間の大半は**LLM側の応答待ち**であり、自作サーバー側の処理は数ミリ秒～十数ミリ秒。
- AIの数にほぼ比例して伸びるため、実運用での体感を考えると **同時に発言させるAIは3体程度が現実的**と考えている。

6. 別PCでの動作検証（Docker化）

ご指摘いただいた「そのサーバーが別のPCで動くかどうか」について、検証を行った。

6.1 検証方法

実際の移設を想定し、**開発環境に依存しないクリーンな状態**を用意して確認した。

1. リポジトリを **ZIP形式でダウンロードした状態**（＝Gitが未インストールのPCと同条件）を再現
2. そこへ **Dockerのみ**を使って起動（Node.js・Gitのインストールは一切行わない）
3. 起動コマンドは **1行のみ**

6.2 結果

- **正常に起動し、3体のAIによる応答まで動作**（所要6.3秒、上記5.2と同一の結果）
- Node.js も Git も不要で、**Dockerさえ入っていれば動く**ことを確認
- Windows / Mac のいずれでも同一のイメージで動作する構成

6.3 意義

- 研究室の他のPCへ移す際、**環境構築で詰まる要素を排除**できる

- 数年後に別の学生が引き継ぐ場合も、**同じ環境を再現して起動できる**
- 過去にWindows機でGit・Node.jsのインストールに詰まった経験を踏まえ、それらを一切不要にする経路を用意した

研究室メンバー向けのセットアップ手順も、この経路を推奨として整備済みである。

7. 「自作」と「OSS利用」の線引き（表現の整理）

先生からご指摘のあった表現の件について、以下の線引きで統一している。

レイヤ	使用技術	区分
LabVIEW側VI	LabVIEW標準（HTTP Client VIs / JSON VIs）	自作VI （標準部品を配線）
実行環境	Docker	OSS（再現性・移植性のため採用）
Webフレームワーク・言語	Next.js / TypeScript / Node.js	OSS（土台）
LLM接続	各社公式SDK	OSS（公式ライブラリ）
LV用API・振り分け・ペルソナ定義・会話制御	本プロジェクトの独自実装	自作（本研究の中核）

- 正確な表現：「**標準的なOSSフレームワークとDockerの上に、LabVIEW-LLM接続インフラを自作した**」
- **Dify / Flowise / LiteLLM / NextChat / OpenWebUI などの既製LLM製品は不使用。** 接続・振り分け・会話制御はすべて独自実装である。

8. 残課題と次フェーズ

8.1 残課題

- **研究室Windows機でのDocker起動**：手順は整備済みだが、実機での実施はこれから
- **応答時間**：AIの数に比例して伸びるため、体感速度の改善余地がある
- **通信の暗号化（HTTPS化）**：医療データを扱う段階では必須。現状はLAN内のHTTP
- **会話ログの蓄積**：患者ごとに分けて時系列で追う仕組みは未着手

8.2 次フェーズ（段階C）で相談したいこと

実装の準備は整っているが、**中身の設計は臨床的な知見が必要**なため、以下についてご意見をいただきたい。

- **発言順の決め方**：どのような場面で、どの役割のAIに話させるべきか
 - **質問の投げ方**：患者から主観（「続けたいか」「頑張る気が出たか」等）を引き出すための問いかけの流れ
 - **役割の設計**：どのような視点のAIを揃えると、評価として意味のある多角性になるか
 - **発言量の調整**：3体が毎回発言すべきか、場面によって絞るべきか
-

9. 総括

先日のミーティングで合意した **マルチエージェント対話システム** について、その中核となる **サーバー側での会話制御** を実装し、**LabVIEW実機で動作を確認した**。

単なる複数AIへの同時問い合わせではなく、**互いの発言を読み合う会話**として成立している点、および次段階のファシリテーターAI導入に向けて**構造的な準備を済ませている**点が今回の成果である。

あわせて、Docker化により **サーバーが別PCでも動くこと** を実証し、本システムが研究室の資産として引き継ぎ可能な形になったことを確認した。